

NEXT.JS

ADVANCED HANDBOOK

Tổng hợp kiến thức Next.js nâng cao - App Router

Cập nhật theo Next.js 16 | 2026

Phạm vi: Tài liệu tập trung vào App Router và các pattern hiện đại. Pages Router chỉ được nhắc đến khi cần so sánh hoặc hỗ trợ migration.

MỤC LỤC

1. Kiến trúc Next.js hiện đại và mental model
2. Advanced Routing Techniques
3. Data Fetching Strategies
4. Caching, Revalidation và Cache Components
5. Performance Optimization Techniques
6. Styling và CSS trong Next.js
7. State Management trong Next.js Applications
8. Authentication & Authorization
9. Testing Next.js Applications
10. Deployment và Hosting
11. Scalability Patterns và Best Practices
12. Internationalization & Localization
13. Redux với Next.js App Router
14. SEO Optimization cho Next.js
15. Performance & Optimization - Audit thực chiến
16. Advanced Next.js Features & Patterns
17. Security, Error Handling và Observability
18. Project Structure, Conventions và Checklist
19. Quick Reference / Cheat Sheet
20. Tài liệu tham khảo chính thức

1. KIẾN TRÚC NEXT.JS HIỆN ĐẠI VÀ MENTAL MODEL

Nền tảng để hiểu đúng App Router trước khi học các kỹ thuật nâng cao.

Next.js là React framework cho full-stack web application. Với App Router, điểm khác biệt lớn nhất so với SPA truyền thống là việc kết hợp React Server Components (RSC), Client Components, streaming, file-system routing, server-side data access và nhiều tầng cache.

Next.js 16: Turbopack là bundler mặc định; Cache Components và mô hình `use cache` trở thành hướng tối ưu quan trọng; `proxy.ts` là convention mới thay cho tên `middleware.ts` trong các luồng mới.

1.1 Server Components vs Client Components

Loại	Chạy ở đâu	Dùng hooks/browser API	Phù hợp
Server Component	Server	Không	Fetch DB/API, render nội dung, giữ secret, giảm JS client
Client Component	Server render + hydrate client	Có	Interactive UI, event handler, local/global client state
Server Function / Server Action	Server	Không trực tiếp	Mutation, form submit, thao tác secret/DB

- **Mặc định là Server Component:** Trong thư mục `app`, component không có `use client` sẽ là Server Component.
- **Đặt boundary hợp lý:** Không đưa `use client` lên root nếu không cần, vì sẽ kéo thêm JavaScript sang client.
- **Data ownership:** Data server nên được fetch gần nơi sử dụng, còn state tương tác UI mới nên sống ở client.
- **Serialization:** Props truyền từ Server sang Client Component phải serializable.

Boundary Server -> Client

```
// app/products/page.tsx - Server Component
export default async function Page() {
  const products = await db.product.findMany()
  return <ProductFilter initialProducts={products} />
}
// ProductFilter.tsx
'use client'
export default function ProductFilter({ initialProducts }) {
  const [query, setQuery] = useState('') // interactive state
}
```

1.2 Rendering model

- **Static rendering:** output có thể được tạo trước và phân phối qua CDN.
- **Dynamic rendering:** render theo request khi phụ thuộc cookies, headers, auth hoặc dữ liệu phải luôn mới.
- **Streaming:** gửi UI từng phần thông qua Suspense thay vì đợi toàn bộ trang.
- **Partial Prerendering / Cache Components:** phối hợp phần cached và phần dynamic để đạt tốc độ tốt mà vẫn giữ tính cá nhân hóa.

Mental model quan trọng Đừng hỏi “trang này là SSR hay CSR?” như SPA cũ. Với App Router, một route có thể kết hợp static shell, cached/dynamic server content và client islands.

2. ADVANCED ROUTING TECHNIQUES

App Router dùng file-system routing. Mỗi folder là một route segment; các file đặc biệt (`page.tsx`, `layout.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx`, `route.ts`, `template.tsx`, `default.tsx`) định nghĩa hành vi của segment.

2.1 Dynamic Segments

Pattern	Ví dụ URL	Ý nghĩa
[id]	/products/42	Một dynamic segment
[...slug]	/docs/a/b/c	Catch-all, bắt >=1 segment
[[...slug]]	/docs hoặc /docs/a/b	Optional catch-all

Next.js hiện đại: `params`/`searchParams` có thể là async APIs

```
// app/products/[id]/page.tsx
export default async function ProductPage({ params }) {
  const { id } = await params
  return <h1>Product {id}</h1>
}
```

2.2 Route Groups

Folder dạng `(group)` dùng để tổ chức route mà không xuất hiện trong URL. Phù hợp cho tách layout theo khu vực public/auth/admin.

```
app/
  (marketing)/
    layout.tsx
    page.tsx // /
    pricing/page.tsx // /pricing
  (dashboard)/
    dashboard/page.tsx // /dashboard
```

2.3 Parallel Routes

Parallel Routes dùng slot `@name` để render nhiều nhánh UI song song trong cùng một layout. Rất phù hợp dashboard, split-view, modal route hoặc UI có nhiều panel độc lập.

```
app/dashboard/
  layout.tsx
  @analytics/page.tsx
  @team/page.tsx

export default function Layout({ children, analytics, team }) {
  return (
    <>
      {children}
      <section>{analytics}</section>
      <aside>{team}</aside>
    </>
  )
}
```

2.4 Intercepting Routes

Intercepting Routes cho phép mở route đích trong context hiện tại, ví dụ click ảnh trong gallery thì mở modal, nhưng refresh trực tiếp URL vẫn hiển thị full page.

- ``(.)`` intercept cùng level
- ``(..)`` lên một level
- ``(..)(..)`` lên hai level
- ``(...)`` từ root app

2.5 Layout vs Template

Đặc điểm	layout.tsx	template.tsx
Giữ state khi navigate	Có	Không, tạo instance mới
Phù hợp	Nav/sidebar, shell ổn định	Reset form/animation/effect theo route
Rerender khi chuyển sibling route	Hạn chế	Có

2.6 Loading, Error và Not Found boundaries

- ``loading.tsx``: Suspense fallback tự động cho route segment.
- ``error.tsx``: Error Boundary ở client; có ``reset()`` để retry.
- ``global-error.tsx``: bắt lỗi ở root layout.
- ``not-found.tsx`` + ``notFound()``: 404 theo segment.
- ``default.tsx``: fallback cho slot của Parallel Routes khi Next.js không có active state phù hợp.

2.7 Rewrites, Redirects và Proxy

Cơ chế	Khi dùng
<code>redirect()/permanentRedirect()</code>	Điều hướng sau auth, mutation hoặc canonical URL
<code>next.config rewrites</code>	Proxy URL nội bộ, legacy backend, vanity URL
<code>next.config redirects</code>	Mapping URL cố định, migration SEO
<code>proxy.ts</code>	Logic request-level: auth gate, locale routing, experiments, header manipulation

Best practice Proxy nên làm việc nhẹ và sớm. Không đặt truy vấn database nặng hoặc business logic phức tạp vào lớp request interception nếu có thể xử lý trong server layer sau đó.

3. DATA FETCHING STRATEGIES

App Router cho phép fetch ngay trong Server Components. Điều này giúp giảm boilerplate API nội bộ và tránh gửi secret/database credentials xuống browser.

3.1 Fetch ở Server Component

```
export default async function Page() {
  const res = await fetch('https://api.example.com/products')
  if (!res.ok) throw new Error('Fetch failed')
  const products = await res.json()
  return <ProductList products={products} />
}
```

- Có thể query DB trực tiếp từ Server Component hoặc server-only data layer.
- Không cần `useEffect` + loading state cho dữ liệu dùng để render lần đầu.
- Không expose token/secret sang client nếu module chỉ chạy server.
- Ưu tiên fetch ở nơi gần component sử dụng để tận dụng streaming và composition.

3.2 Parallel fetching và tránh waterfall

```
const productsPromise = getProducts()
const categoriesPromise = getCategories()
const [products, categories] = await Promise.all([
  productsPromise,
  categoriesPromise,
])
```

Waterfall không phải luôn xấu Nếu request B thực sự cần kết quả của A thì tuần tự là đúng. Tối ưu chỉ những waterfall không có dependency.

3.3 Streaming với Suspense

```
import { Suspense } from 'react'

export default function Page() {
  return (
    <main>
      <FastHeader />
      <Suspense fallback={<ChartSkeleton />}>
        <SlowRevenueChart />
      </Suspense>
    </main>
  )
}
```

Ưu điểm: TTFB và perceived performance tốt hơn; phần nhanh hiển thị sớm. Hãy đặt Suspense boundary theo “user-perceived unit” thay vì bọc từng component nhỏ.

3.4 Client-side fetching

Dùng khi dữ liệu phụ thuộc tương tác client, polling/realtime, API browser-only hoặc không cần SEO. Có thể dùng SWR, React Query/TanStack Query hoặc RTK Query.

Strategy	Khi phù hợp	Lưu ý
Server Component fetch	Data render ban đầu, SEO, DB access	Ưu tiên mặc định

Route Handler + client fetch	Public API, browser fetch	Thêm network hop
SWR/TanStack Query	Client cache, polling, optimistic UI	Chỉ client state/data
Server Action	Mutation từ UI	Validate + authorize ở server
Direct DB query	Server-only full-stack app	Không gọi từ client

3.5 Route Handlers

```
// app/api/products/route.ts
import { NextResponse } from 'next/server'

export async function GET() {
  const products = await getProducts()
  return NextResponse.json(products)
}

export async function POST(req: Request) {
  const body = await req.json()
  // validate + auth + mutate
  return NextResponse.json({ ok: true }, { status: 201 })
}
```

Route Handlers phù hợp webhooks, public/private APIs, third-party integrations, file responses. Nếu chỉ Server Component cần DB thì thường không cần tạo API trung gian.

3.6 Mutations với Server Actions

```
// app/actions.ts
'use server'
import { revalidatePath } from 'next/cache'

export async function createProduct(formData: FormData) {
  const input = parseAndValidate(formData)
  await authorizeCurrentUser()
  await db.product.create({ data: input })
  revalidatePath('/products')
}
```

- Validate input bằng Zod/Valibot hoặc schema tương đương.
- Authorization phải kiểm tra trong chính action; không tin UI đã ấn button.
- Sau mutation, revalidate cache/path/tag phù hợp.
- Tránh trả stack trace hoặc database error thô về client.

4. CACHING, REVALIDATION VÀ CACHE COMPONENTS

Caching là phần dễ nhầm nhất của App Router vì có nhiều lớp: request/data cache, server rendering cache, client router cache và các API cache mới. Với Next.js 16, tư duy nên chuyển sang cache explicit hơn thay vì giả định mọi `fetch` đều tự động cached.

4.1 Khi nào nên cache?

Loại dữ liệu	Cache?	Chiến lược
Danh mục sản phẩm công khai	Có	Cache + tag + revalidate khi admin cập nhật
Profile người đang đăng nhập	Thường không dùng shared cache	Dynamic/request scoped
Feature flags ít đổi	Có	Cache theo thời gian hoặc tag
Giỏ hàng hiện tại	Không shared	Cookie/session/client state
Bài viết CMS	Có	Cache + on-demand revalidation

4.2 `use cache` và Cache Components

```
// next.config.ts
const nextConfig = {
  cacheComponents: true,
}
export default nextConfig

// app/lib/products.ts
'use cache'
import { cacheLife, cacheTag } from 'next/cache'

export async function getCatalog() {
  cacheLife('hours')
  cacheTag('catalog')
  return db.product.findMany({ where: { published: true } })
}
```

Mục tiêu: đánh dấu rõ phần logic có thể cache; `cacheLife` mô tả lifetime/profile, `cacheTag` cho phép invalidation theo domain entity.

4.3 Revalidation

```
'use server'
import { revalidatePath, revalidateTag } from 'next/cache'

export async function updateProduct(id: string, data: ProductInput) {
  await db.product.update({ where: { id }, data })
  revalidateTag('catalog')
  revalidatePath(`/products/${id}`)
}
```

- `revalidatePath`: invalidates theo URL/path.
- `revalidateTag`: invalidates nhóm dữ liệu có cùng tag.
- `updateTag` (Next.js 16): hữu ích khi muốn update/invalidate tag trong mutation flow với semantics mới.
- Không revalidate toàn site nếu chỉ một entity thay đổi.

4.4 Dynamic APIs và cache boundary

Các API request-specific như ``cookies()``, ``headers()``, auth session hoặc search params có thể khiến phần UI phải dynamic. Tối ưu bằng cách cô lập chúng ở subtree nhỏ, còn phần tĩnh/cached giữ ở ngoài.

Pattern Static/cached shell ở ngoài + dynamic island trong `<Suspense>` thường tốt hơn biến cả page thành dynamic chỉ vì một widget cá nhân hóa nhỏ.

5. PERFORMANCE OPTIMIZATION TECHNIQUES

5.1 Giảm JavaScript gửi xuống client

- Giữ component là Server Component nếu không cần state, effect hoặc event handler.
- Đẩy `use client` xuống thấp nhất có thể.
- Không import thư viện lớn vào Client Component chỉ để xử lý server-side data.
- Dùng dynamic import/lazy loading cho editor, chart, map, modal nặng.

```
import dynamic from 'next/dynamic'

const HeavyChart = dynamic(() => import('./HeavyChart'), {
  loading: () => <ChartSkeleton />,
})
```

5.2 Image optimization

```
import Image from 'next/image'

<Image
  src={product.image}
  alt={product.name}
  width={800}
  height={600}
  sizes="(max-width: 768px) 100vw, 50vw"
  priority={isHero}
/>
```

- Cung cấp kích thước để tránh CLS.
- Dùng `sizes` đúng với responsive layout.
- Chỉ `priority` cho LCP/hero image quan trọng.
- Thiết lập `remotePatterns` rõ ràng cho external images.

5.3 Font optimization

```
import { Inter } from 'next/font/google'
const inter = Inter({ subsets: ['latin'], display: 'swap' })
```

`next/font` self-host font asset, tránh request font ngoài runtime và giúp giảm layout shift.

5.4 Script optimization

- Dùng `next/script` với strategy phù hợp: `beforeInteractive`, `afterInteractive`, `lazyOnload`.
- Không load analytics/chat widget trước nội dung chính nếu không cần.
- Audit third-party scripts vì chúng thường chiếm main-thread time lớn.

5.5 Code splitting và bundle analysis

- Mỗi route được code-split tự động, nhưng shared client imports vẫn có thể tạo bundle lớn.
- Tránh barrel file không cần thiết ở thư viện lớn nếu tree-shaking kém.
- Phân tích bundle khi client JS tăng bất thường.
- Ưu tiên package ESM, nhỏ và server-compatible.

5.6 React Compiler và memoization

Next.js 16 có tích hợp React Compiler ổn định. Tuy nhiên, không nên “memo hóa mọi thứ” thủ công. `useMemo`/`useCallback` chỉ nên dùng khi có lý do đo được hoặc cần reference stability cho dependency/API.

5.7 Prefetching và navigation

`<Link>` hỗ trợ client-side navigation và prefetch. Với route cực nặng hoặc danh sách hàng nghìn link, cần cân nhắc prefetch behavior và virtualization thay vì để browser/network bị bão request.

Nguyên tắc Performance optimization tốt nhất thường đến từ kiến trúc: ít client JS, fetch song song, cache đúng scope và streaming đúng boundary - không phải từ micro-optimization.

6. STYLING VÀ CSS TRONG NEXT.JS

6.1 Global CSS

```
// app/layout.tsx
import './globals.css'

export default function RootLayout({ children }) {
  return <html lang="vi"><body>{children}</body></html>
}
```

6.2 CSS Modules

```
/* Button.module.css */
.primary { padding: .75rem 1rem; border-radius: .5rem; }

// Button.tsx
import styles from './Button.module.css'
export function Button() {
  return <button className={styles.primary}>Save</button>
}
```

CSS Modules scope class theo component và được Next.js xử lý/code-split.

6.3 Tailwind CSS

- Phù hợp design system utility-first và tốc độ triển khai UI.
- Tách reusable component thay vì copy class dài ở quá nhiều nơi.
- Dùng `clsx`/`cn` cho conditional class.
- Đặt design token (color, spacing, radius) thống nhất.

6.4 Sass và CSS-in-JS

Next.js hỗ trợ Sass. Với CSS-in-JS, cần kiểm tra mức độ tương thích với Server Components và streaming. Những library phụ thuộc runtime style injection thường cần Client Component/provider hoặc cấu hình registry.

6.5 Styling strategy cho dự án lớn

Layer	Khuyến nghị
Design tokens	CSS variables hoặc theme config
Primitives	Button, Input, Dialog, Card
Feature components	Composition từ primitives
Page-specific styles	Giữ gần route/feature
Global styles	Reset, typography, tokens - tối thiểu

Best practice: giữ `globals.css` tối thiểu; tránh dùng nó như nơi chứa toàn bộ style dự án để giảm cascade conflict.

7. STATE MANAGEMENT TRONG NEXT.JS APPLICATIONS

Trong App Router, không nên mặc định dùng global state library cho mọi loại state. Hãy phân loại state trước.

Loại state	Nơi nên đặt
Server data	Server Component / cache / data layer
URL state	params, searchParams, pathname
Form state	Server Action + `useActionState` / form library
Local interactive state	useState/useReducer
Shared client UI state	Context/Zustand/Redux/Jotai...
Persistent client state	localStorage/IndexedDB + hydration strategy

7.1 URL là state manager mạnh

Filter, sort, pagination, tab có tính shareable/bookmarkable nên ưu tiên `searchParams` hơn global store.

```
// /products?category=shoes&page=2
export default async function Page({ searchParams }) {
  const params = await searchParams
  const page = Number(params.page ?? 1)
  const category = params.category ?? 'all'
  // fetch based on URL state
}
```

7.2 React Context

Dùng cho theme, UI preferences, feature-local shared state. Provider phải là Client Component nhưng children có thể vẫn chứa Server Components thông qua composition.

7.3 Zustand/Jotai/Redux

- Tốt cho mutable client state cần chia sẻ rộng.
- Không dùng để mirror toàn bộ server data nếu Server Components/cache đã giải quyết được.
- Cẩn thận hydration khi state đọc localStorage.
- Scope store theo feature/route khi có thể.

Decision rule Nếu state có thể biểu diễn bằng URL, server data hoặc local component state, hãy dùng cách đó trước. Global store là công cụ cuối cùng chứ không phải mặc định.

8. AUTHENTICATION & AUTHORIZATION

Authentication trả lời “ai đang đăng nhập?”, Authorization trả lời “người đó được phép làm gì?”. Trong Next.js full-stack, cả hai phải được enforce ở server, không chỉ bằng cách ẩn UI.

8.1 Authentication patterns

- Auth library (Auth.js/NextAuth, Clerk, Supabase Auth, Firebase Auth...) giúp giảm rủi ro tự triển khai session/OAuth.
- Session thường lưu bằng encrypted/signed cookie hoặc session ID server-side.
- OAuth/OIDC phù hợp social/enterprise identity.
- Credentials auth cần password hashing mạnh (Argon2/bcrypt), rate limit và account protection.

8.2 Route protection với Proxy

```
// proxy.ts - simplified example
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function proxy(req: NextRequest) {
  const hasSession = Boolean(req.cookies.get('session'))
  if (!hasSession && req.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', req.url))
  }
  return NextResponse.next()
}

export const config = { matcher: ['/dashboard/:path*'] }
```

Quan trọng Kiểm tra cookie tồn tại ở Proxy chỉ là gate ban đầu. Authorization thực sự vẫn phải kiểm tra trong Server Component, Route Handler hoặc Server Action trước khi đọc/ghi dữ liệu nhạy cảm.

8.3 DAL authorization

```
// lib/authz.ts
import 'server-only'

export async function requireProjectAccess(projectId: string) {
  const session = await getSession()
  if (!session) throw new Error('UNAUTHENTICATED')

  const member = await db.projectMember.findUnique({
    where: { projectId_userId: { projectId, userId: session.user.id } }
  })
  if (!member) throw new Error('FORBIDDEN')
  return { session, member }
}
```

8.4 RBAC và ABAC

- **RBAC**: admin/editor/viewer.
- **ABAC**: quyền dựa trên thuộc tính user/resource.
- **Ownership**: resource.ownerId === user.id.
- **Hybrid**: role + ownership + trạng thái resource.
- Không trust `role` từ client và không expose session secret/token vào Client Component.
- Dùng CSRF protection theo cơ chế auth/library khi cần; rotate secret và cấu hình cookie `httpOnly`, `secure`, `sameSite` phù hợp.

9. TESTING NEXT.JS APPLICATIONS

Một test strategy tốt nên có nhiều tầng thay vì cố dùng E2E cho tất cả.

Tầng	Tool phổ biến	Test gì
Unit	Vitest/Jest	Pure functions, reducers, validators
Component	React Testing Library	Client components, form behavior
Integration	Vitest/Jest + test DB/MSW	Data layer, route handlers
E2E	Playwright/Cypress	Critical user flows, routing, auth
Static	TypeScript, ESLint	Type/lint regressions

9.1 Unit test

```
import { describe, it, expect } from 'vitest'
import { calculateTotal } from './pricing'

describe('calculateTotal', () => {
  it('applies discount', () => {
    expect(calculateTotal(100, 0.1)).toBe(90)
  })
})
```

9.2 Component test

Client Components test gần như React thường. Với async Server Components, integration/E2E thường thực tế hơn vì test environment cho RSC có giới hạn tùy tool.

9.3 E2E Playwright

```
import { test, expect } from '@playwright/test'

test('user can sign in and open dashboard', async ({ page }) => {
  await page.goto('/login')
  await page.getByLabel('Email').fill('user@example.com')
  await page.getByLabel('Password').fill('secret')
  await page.getByRole('button', { name: 'Sign in' }).click()
  await expect(page).toHaveURL(/dashboard/)
})
```

9.4 Test database và mocks

- Unit: mock boundary, không mock implementation sâu.
- Integration: dùng database test/transaction rollback nếu có thể.
- E2E: seed dữ liệu deterministic.
- API third-party: MSW hoặc stub server để tránh flaky network.
- Không test chỉ snapshot markup; ưu tiên behavior.

CI: chạy `typecheck + lint + unit` cho mọi PR; E2E cho critical flows và lưu trace/screenshot khi fail.

10. DEPLOYMENT VÀ HOSTING

10.1 Vercel

Vercel là nền tảng tích hợp sâu với Next.js: preview deployments, serverless/edge runtime, image optimization, cache/revalidation và observability. Đây thường là đường triển khai ít cấu hình nhất.

10.2 Self-hosting Node.js

```
npm run build
npm run start
```

Có thể chạy sau reverse proxy (Nginx/Cloudflare/ALB), container hoặc VM. Cần tự quản lý scaling, cache/CDN, logging, image optimization và secrets.

10.3 Docker / standalone output

```
// next.config.ts
const nextConfig = { output: 'standalone' }
export default nextConfig
```

Standalone output giúp image/container nhỏ hơn vì chỉ copy runtime files cần thiết.

10.4 Static export

```
// next.config.ts
const nextConfig = { output: 'export' }
export default nextConfig
```

Chỉ phù hợp app không cần server features tại runtime. Các khả năng phụ thuộc request/server như dynamic server rendering, Server Actions, một số image optimization hoặc Route Handlers động sẽ bị hạn chế.

10.5 Environment variables

- Biến không có prefix `NEXT\_PUBLIC\_` giữ server-only (nếu không bị đưa vào client code).
- `NEXT\_PUBLIC\_\*` được bundle/expose cho browser - không chứa secret.
- Tách dev/staging/prod và rotate secrets.
- Không commit `.env.local`.

10.6 Deployment checklist

- Build production locally/CI trước khi deploy.
- Kiểm tra database migrations và backward compatibility.
- Set health check/readiness nếu self-host.
- Log structured + error tracking.
- Preview deployment cho PR.
- Rollback strategy rõ ràng.

11. SCALABILITY PATTERNS VÀ BEST PRACTICES

11.1 Stateless application layer

Để scale horizontally, không lưu session/user state trong memory của một process. Dùng database, Redis/KV hoặc signed cookie tùy use case.

11.2 Data Access Layer (DAL)

```
// lib/data/products.ts
import 'server-only'

export async function listProducts(input: ProductQuery) {
  const where = buildProductWhere(input)
  return db.product.findMany({ where, take: 50 })
}
```

DAL giúp gom authorization, query optimization, caching và domain rules; tránh để page/component biết quá nhiều chi tiết database.

11.3 Bounded context / feature modules

```
src/
  app/
  features/
    auth/
    catalog/
    checkout/
  lib/
    db/
    observability/
  components/
  ui/
```

Khi codebase lớn, tổ chức theo domain/feature thường scale team tốt hơn kiểu thư mục chỉ theo kỹ thuật (`components`, `hooks`, `utils`) chứa hàng trăm file lẫn lộn.

11.4 Database scalability

- Index theo query thật, tránh N+1.
- Pagination bằng cursor cho dữ liệu lớn.
- Connection pooling phù hợp serverless.
- Read replicas cho read-heavy khi cần.
- Cache dữ liệu đọc nhiều nhưng invalidation rõ ràng.
- Không fetch toàn bộ table rồi filter ở Node.js.

11.5 Background jobs và event-driven

Email, image processing, reporting, webhook retry hoặc công việc > request latency budget nên đưa vào queue/job system. Server Action/Route Handler chỉ enqueue rồi trả response nhanh.

11.6 Rate limiting và abuse protection

- Rate limit login, password reset, public APIs, expensive search.
- Có thể rate limit tại CDN/WAF, proxy hoặc data layer.
- Dùng idempotency key cho mutation tài chính/đơn hàng.

12. INTERNATIONALIZATION & LOCALIZATION

Internationalization (i18n) là kiến trúc hỗ trợ nhiều locale; localization (l10n) là nội dung/format cụ thể cho từng locale.

12.1 Locale trong URL

```
app/
  [lang]/
    layout.tsx
    page.tsx
    products/page.tsx
```

URL locale (`/vi/...`, `/en/...`, `/ja/...`) để SEO, share và cache. Có thể dùng Proxy để detect `Accept-Language` và redirect locale mặc định.

12.2 Load dictionary trên server

```
const dictionaries = {
  en: () => import('./dictionaries/en.json').then(m => m.default),
  vi: () => import('./dictionaries/vi.json').then(m => m.default),
}

export async function getDictionary(locale: 'en' | 'vi') {
  return dictionaries[locale]()
}
```

- Chỉ load dictionary locale hiện tại.
- Format date/number/currency bằng `Intl.\*`.
- Tách translation key ổn định khỏi copy text trực tiếp.
- Cân nhắc `next-intl` hoặc library chuyên dụng cho pluralization, ICU message, routing.

12.3 SEO đa ngôn ngữ

- `html lang` đúng locale.
- Canonical theo locale.
- `alternates.languages` / hreflang cho các phiên bản ngôn ngữ.
- Localized metadata title/description/OG image khi cần.

Không chỉ dịch chữ Locale có thể ảnh hưởng currency, timezone, date order, plural rules, address/phone format, typography và cả layout RTL.

13. REDUX VỚI NEXT.JS (APP ROUTER)

Redux vẫn hữu ích cho globally shared mutable client state, nhưng App Router yêu cầu setup khác SPA truyền thống.

Nguyên tắc chính thức Không tạo global singleton store dùng chung giữa server requests; RSC không đọc/ghi Redux store; chỉ Client Components tương tác với Redux context. Redux nên chứa mutable client data, không phải mọi server data.

13.1 Cài đặt

```
npm install @reduxjs/toolkit react-redux
```

13.2 Tạo store factory

```
// lib/store.ts
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from '../features/counter/counterSlice'

export const makeStore = () => configureStore({
  reducer: { counter: counterReducer },
})

export type AppStore = ReturnType<typeof makeStore>
export type RootState = ReturnType<AppStore['getState']>
export type AppDispatch = AppStore['dispatch']
```

13.3 StoreProvider client component

```
// app/StoreProvider.tsx
'use client'
import { useRef } from 'react'
import { Provider } from 'react-redux'
import { makeStore, type AppStore } from '@lib/store'

export default function StoreProvider({ children }) {
  const storeRef = useRef<AppStore | null>(null)
  if (!storeRef.current) storeRef.current = makeStore()
  return <Provider store={storeRef.current}>{children}</Provider>
}
```

13.4 Typed hooks

```
// lib/hooks.ts
'use client'
import { useDispatch, useSelector, useStore } from 'react-redux'
import type { AppDispatch, AppStore, RootState } from './store'

export const useAppDispatch = useDispatch.withTypes<AppDispatch>()
export const useAppSelector = useSelector.withTypes<RootState>()
export const useAppStore = useStore.withTypes<AppStore>()
```

13.5 Provider nên đặt ở đâu?

- Đặt Provider ở layout nhỏ nhất cần store; chỉ đặt root khi state thật sự toàn app. Provider là Client Component nhưng Server Components vẫn có thể được truyền qua `children`.

13.6 RTK Query

Theo hướng dẫn Redux hiện tại, RTK Query phù hợp chủ yếu cho client-side fetching. Server data trong App Router nên fetch trực tiếp từ async RSC/data layer và tận dụng Next.js cache/streaming.

Anti-pattern Không export `const store = configureStore(...)` ở module global trong server-rendered App Router. Store có thể bị chia sẻ giữa nhiều request và gây data leakage/hydration issues.

14. SEO OPTIMIZATION CHO NEXT.JS

14.1 Metadata API

```
// app/products/[id]/page.tsx
import type { Metadata } from 'next'

export async function generateMetadata({ params }): Promise<Metadata> {
  const { id } = await params
  const product = await getProduct(id)
  return {
    title: product.name,
    description: product.summary,
    alternates: { canonical: `/products/${id}` },
    openGraph: {
      title: product.name,
      images: [product.ogImage],
    },
  },
}
```

14.2 Sitemap và robots

```
// app/sitemap.ts
import type { MetadataRoute } from 'next'

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const products = await getPublishedProducts()
  return products.map(p => ({
    url: `https://example.com/products/${p.slug}`,
    lastModified: p.updatedAt,
  }))
}

// app/robots.ts
export default function robots(): MetadataRoute.Robots {
  return { rules: { userAgent: '*', allow: '/' }, sitemap: 'https://example.com/sitemap.xml' }
}
```

14.3 Structured data (JSON-LD)

Dùng schema.org phù hợp như Product, Article, BreadcrumbList, Organization. Serialize cẩn thận để tránh XSS khi chèn JSON-LD.

14.4 Technical SEO checklist

- SSR/static HTML có nội dung chính, không để toàn trang chỉ render sau client fetch.
- Canonical URL đúng.
- Unique title/description.
- OG/Twitter metadata.
- Sitemap, robots, hreflang.
- Semantic HTML + heading hierarchy.
- Core Web Vitals tốt.
- 404/redirect status code đúng.
- Pagination/filter không tạo vô hạn crawl space.

Kết luận SEO: rendering strategy, crawlability, internal linking, performance và content quality thường quan trọng hơn việc nhồi thêm meta tags.

15. PERFORMANCE & OPTIMIZATION - AUDIT THỰC CHIẾN

Chương này biến performance thành quy trình đo -> tìm bottleneck -> tối ưu -> đo lại.

15.1 Core Web Vitals

Metric	Ý nghĩa	Nguồn lỗi thường gặp
LCP	Nội dung lớn nhất hiển thị	Hero image, font, server latency
INP	Responsiveness tương tác	JS client lớn, long tasks, third-party
CLS	Layout shift	Image không có size, font swap, ads/injected UI

15.2 Audit server

- TTFB cao? đo DB/query/external API.
- Waterfall? chạy parallel hoặc cache.
- Render cả page dynamic vì một widget? cô lập dynamic subtree.
- Query lặp lại? request memoization/cache/data layer.
- Serverless cold start? giảm dependency, chọn runtime phù hợp.

15.3 Audit client

- Xem JS bundle per route.
- Tìm Client Component boundary quá cao.
- Tìm chart/editor/date library nặng.
- Tìm hydration mismatch gây rerender.
- Tìm third-party scripts.
- Virtualize list/table lớn.

15.4 Audit network

- Nén Brotli/Gzip.
- Cache-Control/CDN hợp lý cho static assets.
- Ảnh đúng format/size.
- Không preload quá nhiều.
- API response chỉ trả fields cần thiết.

15.5 Optimization order

1. Đo RUM/Lighthouse/trace.
2. Tìm bottleneck lớn nhất.
3. Fix kiến trúc (rendering/data/cache) trước.
4. Fix asset và bundle.
5. Fix micro-interactions.
6. Đo lại và ghi regression budget.

Performance budget gợi ý Theo dõi client JS theo route, LCP/INP p75, server response p95, query latency p95 và image weight. Budget giúp ngăn performance suy giảm dần qua nhiều PR.

16. ADVANCED NEXT.JS FEATURES & PATTERNS

16.1 Server-only modules

```
import 'server-only'

export async function getSecretData() {
  return fetch(process.env.PRIVATE_API_URL!, {
    headers: { Authorization: `Bearer ${process.env.PRIVATE_TOKEN}` }
  })
}
```

`server-only` giúp phát hiện sớm nếu module chứa secret bị import nhầm vào client graph.

16.2 Composition pattern

Client Component có thể nhận Server Component qua `children`/props, giúp giữ server rendering bên trong interactive shell.

```
// Client shell
'use client'
export function Modal({ children }) {
  return <dialog open>{children}</dialog>
}

// Server page
<Modal>
  <ServerRenderedProductDetails />
</Modal>
```

16.3 Optimistic UI

Dùng `useOptimistic`, Server Actions hoặc client cache mutation để phản hồi ngay; rollback khi server reject. Phù hợp like, todo, quantity, rename - nhưng với thanh toán cần xác nhận server chặt chẽ.

16.4 Form actions và progressive enhancement

Form gọi Server Action có thể hoạt động ngay cả khi JS chưa hydrate trong nhiều tình huống, đem lại progressive enhancement tốt.

16.5 Instrumentation

Dùng instrumentation hooks/OpenTelemetry để khởi tạo tracing, metrics hoặc monitoring server. Correlation ID giúp nối request -> DB -> external API -> log.

16.6 Runtime choice: Node.js vs Edge

Node.js Runtime	Edge Runtime
Tương thích npm/Node APIs rộng	Latency gần user, runtime nhẹ
Phù hợp DB drivers, SDK nặng	Hạn chế Node APIs/package native
Dễ self-host	Phù hợp request filtering/simple compute

Không chọn Edge chỉ vì “nghe nhanh”; hãy xem dependency compatibility, data locality và cold start/latency thực tế.

16.7 Build adapters và platform integration

Next.js 16 có Build Adapters API ở trạng thái alpha, hướng tới tùy biến build cho platform. Với production, feature alpha cần đánh giá rủi ro/version pinning trước khi phụ thuộc sâu.

16.8 View Transitions / React 19.x features

Các tính năng React mới có thể xuất hiện sớm trong App Router. Luôn kiểm tra version compatibility trước khi dùng production, nhất là API canary/experimental.

17. SECURITY, ERROR HANDLING VÀ OBSERVABILITY

17.1 Input validation

- Mọi input từ form, URL, header, cookie, webhook đều untrusted.
- Validate type/format/range bằng schema.
- Encode/escape output đúng context.
- Parameterized query/ORM để chống SQL injection.

17.2 Server Actions security

- Action là public attack surface theo nghĩa người dùng có thể cố gọi nó; luôn authenticate + authorize.
- Không tin hidden input.
- Rate limit mutation nhạy cảm.
- Không trả object chứa secret hoặc internal error.

17.3 Error boundaries

- Expected error: trả validation/result state thân thiện.
- Unexpected error: throw để error boundary/monitoring xử lý.
- Không dùng exception cho mọi business validation.
- Gắn request ID để trace production.

17.4 Logging

Log structured JSON ở server với fields như requestId, userId hash/internal id, route, duration, status, errorCode. Tránh log token/password/PII không cần thiết.

17.5 CSP và security headers

- Content-Security-Policy để hạn chế XSS impact.
- HSTS khi HTTPS production.
- X-Content-Type-Options, Referrer-Policy, Permissions-Policy phù hợp.
- CORS chỉ mở origin/method/header cần thiết cho public APIs.

Patch discipline Framework full-stack có thể có security advisory. Theo dõi Next.js/React security release và cập nhật patch version sớm, đặc biệt với RSC/server features.

18. PROJECT STRUCTURE, CONVENTIONS VÀ CHECKLIST

18.1 Cấu trúc gợi ý

```
src/  
  app/  
    (public)/  
    (auth)/  
    dashboard/  
    api/  
  components/  
  ui/  
  features/  
    catalog/  
    checkout/  
    account/  
  lib/  
    db/  
    auth/  
    cache/  
    observability/  
  styles/  
  tests/
```

18.2 Naming

- Component: PascalCase.
- Route folders: lowercase/kebab-case.
- Server actions: verb-first (`createOrder`, `updateProfile`).
- Data queries: intent rõ (`getProductBySlug`, `listOrdersForUser`).
- Không dùng `utils.ts` như thùng rác; tách theo domain.

18.3 App Router checklist

- Component nào thật sự cần `use client`?
- Data có thể fetch server trực tiếp không?
- Có waterfall không?
- Cache scope/tag đúng chưa?
- Mutation đã validate + authorize + revalidate chưa?
- Route có loading/error/not-found UX chưa?
- Metadata/SEO đầy đủ chưa?
- Client bundle có thư viện nặng không?
- Auth được enforce ở server chưa?
- Test critical flow chưa?
- Log/monitoring đủ để debug production chưa?

19. QUICK REFERENCE / CHEAT SHEET

Mục tiêu	API / Convention
Page	page.tsx
Shared layout	layout.tsx
Reset-on-navigation layout	template.tsx
Loading boundary	loading.tsx / Suspense
Error boundary	error.tsx
404	not-found.tsx / notFound()
API endpoint	route.ts
Request interception	proxy.ts
Dynamic param	[id]
Catch all	[...slug]
Route group	(group)
Parallel slot	@slot
Server mutation	Server Action / Server Function
Path invalidation	revalidatePath
Tag invalidation	revalidateTag / updateTag
Explicit cache	use cache + cacheLife/cacheTag
Metadata	metadata / generateMetadata
Sitemap	sitemap.ts
Robots	robots.ts
Image	next/image
Font	next/font
Client navigation	next/link
Redirect	redirect / permanentRedirect

19.1 Khi nào dùng cái gì?

Tình huống	Giải pháp ưu tiên
Fetch DB để render page	Server Component + DAL
User bấm submit form	Server Action
Third-party webhook	Route Handler
Filter/pagination shareable	searchParams
Modal route có URL riêng	Intercepting + Parallel Routes

Dashboard nhiều panel tải khác nhau	Suspense + parallel fetch
Catalog ít đổi	Cache + tag
Personalized widget	Dynamic subtree
Global mutable client workflow	Redux/Zustand/Context
SEO product pages	Server render + metadata + sitemap + structured data

20. TÀI LIỆU THAM KHẢO CHÍNH THỨC

Các nguồn nên ưu tiên khi tra cứu vì Next.js thay đổi nhanh:

- Next.js Documentation: <https://nextjs.org/docs>
- Next.js Learn - App Router: <https://nextjs.org/learn/dashboard-app>
- Next.js Blog / Releases: <https://nextjs.org/blog>
- Redux Toolkit + Next.js App Router: <https://redux.js.org/usage/nextjs>
- React Documentation: <https://react.dev>
- Vercel Documentation: <https://vercel.com/docs>
- Auth.js Documentation: <https://authjs.dev>

Versioning note Tài liệu này lấy Next.js 16/App Router làm baseline. Khi dùng dự án có version khác, hãy kiểm tra migration guide và API reference đúng version trước khi áp dụng code.

Gợi ý lộ trình ôn tập

1. Nắm Server vs Client Components + routing conventions.
2. Thực hành fetch server, Suspense, Server Actions.
3. Hiểu cache/revalidation và đo performance.
4. Xây auth + authorization trong DAL.
5. Thêm testing, SEO, i18n và deployment.
6. Cuối cùng mới đưa Redux/global state vào những phần thực sự cần.